

Tarea 5.4.- Chatbot con Machine Learning e Integración web.

En esta tarea, construiremos un chatbot que utiliza aprendizaje automático para clasificar intenciones y se integrará con una aplicación web sencilla usando Flask.

Introducción

Flask es un microframework de desarrollo web para Python. Su principal función es ayudarte a crear aplicaciones web (como sitios web, APIs REST o interfaces para tus modelos de IA) de forma rápida, sencilla y flexible.

Características clave de Flask:

- Ligerio: Solo incluye lo esencial para empezar, sin funcionalidades innecesarias.
- Flexible: Tú decides qué componentes añadir (plantillas, bases de datos, autenticación, etc.).
- Fácil de usar: Su curva de aprendizaje es muy baja, ideal para quienes ya saben Python.
- Popular en prototipado rápido y proyectos de IA: Se usa mucho para exponer modelos de machine learning o chatbots como servicios web.

Ejemplo mínimo de Flask:

```
from flask import Flask
app = Flask(__name__)
@app.route('/')
def home():
    return "¡Hola, mundo desde Flask!"

if __name__ == "__main__":
    app.run()
```

Al ejecutar este código, se levanta un servidor web local y si visitas <http://127.0.0.1:5000/>, verás el mensaje “¡Hola, mundo desde Flask!”.

Actividad guiada.

Crea un entorno de desarrollo en python y realiza los siguientes pasos:

Paso 1: Preparación del entorno

```
python -m venv env
cd env\Scripts\
activate

pip install flask scikit-learn spacy
python -m spacy download en_core_web_sm
```

Paso 2: Dataset de entrenamiento

```
import json
import random

# Intenciones
intents = {
    "intents": [
        {"tag": "greeting", "patterns": ["Hi", "Hey", "Hello"], "responses":
["Hello!", "Hi there!"]},
        {"tag": "goodbye", "patterns": ["Bye", "See you"], "responses":
["Goodbye!", "See you soon!"]},
        {"tag": "thanks", "patterns": ["Thanks", "Thank you"], "responses":
["You're welcome!"]}
    ]
}
```

Paso 3: Procesamiento de datos y entrenamiento

```
import spacy
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

nlp = spacy.load("en_core_web_sm")

X, y = [], []
for intent in intents["intents"]:
    for pattern in intent["patterns"]:
        X.append(pattern)
        y.append(intent["tag"])

vectorizer = CountVectorizer()
X_vec = vectorizer.fit_transform(X)
model = MultinomialNB()
model.fit(X_vec, y)
```

Paso 4: Función de predicción

```
def predict_class(text):
    vec = vectorizer.transform([text]).toarray()
    pred = model.predict(vec)[0]
    return pred
```

Paso 5: Selección de respuesta

```
def get_response(intent_tag):  
    for intent in intents["intents"]:  
        if intent["tag"] == intent_tag:  
            return random.choice(intent["responses"])  
    return "I don't understand."
```

Paso 6: Integrar con Flask

```
from flask import Flask, request, jsonify, render_template_string  
  
# Interfaz HTML  
HTML = """  
<!DOCTYPE html>  
<html>  
<head>  
    <title>Chatbot</title>  
    <style>  
        body { font-family: Arial, sans-serif; max-width: 600px; margin: 40px  
auto; padding: 0 20px; }  
        h2 { color: #333; }  
        #chat {  
            border: 1px solid #ccc;  
            border-radius: 8px;  
            padding: 10px;  
            height: 350px;  
            overflow-y: auto;  
            margin-bottom: 10px;  
            background: #f9f9f9;  
        }  
        #chat p { margin: 6px 0; }  
        .user { color: #0066cc; }  
        .bot { color: #228B22; }  
        #input-area { display: flex; gap: 8px; }  
        #msg { flex: 1; padding: 8px; border: 1px solid #ccc; border-radius: 6px;  
font-size: 14px; }  
        button {  
            padding: 8px 16px;  
            background: #0066cc;  
            color: white;  
            border: none;  
            border-radius: 6px;  
            cursor: pointer;  
            font-size: 14px;  
        }  
        button:hover { background: #0052a3; }  
    </style>  
</head>  
<body>  
    <h2>Chatbot</h2>  
    <div id="chat"></div>  
    <div id="input-area">  
        <input id="msg" type="text" placeholder="Type a message...">  
        <button onclick="send()">Send</button>  
    </div>  
<script>
```

```
async function send() {
  const input = document.getElementById("msg");
  const msg = input.value.trim();
  if (!msg) return;

  const chat = document.getElementById("chat");
  chat.innerHTML += `

<b>You:</b> ${msg}</p>`;
  input.value = "";

  const res = await fetch("/chat", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ message: msg })
  });

  const data = await res.json();
  chat.innerHTML += `

<b>Bot:</b> ${data.response}</p>`;
  chat.scrollTop = chat.scrollHeight;
}

document.getElementById("msg").addEventListener("keypress", e => {
  if (e.key === "Enter") send();
});
</script>
</body>
</html>
"""

# Flask
app = Flask(__name__)

@app.route("/")
def index():
  return render_template_string(HTML)

@app.route("/chat", methods=["POST"])
def chat():
  data = request.get_json()
  user_message = data.get("message", "")
  intent = predict_class(user_message)
  response = get_response(intent)
  return jsonify({"response": response})

if __name__ == "__main__":
  app.run(debug=True)


```